

Variance reduction using stochastic approximation

This writeup is meant to be read together with the codes in this directory.

General setup

Use Monte Carlo sampling to estimate

$$A = \mathbb{E}_{f_0}[V(X)] = \mathbb{E}_0[V(X)] = \int V(x) f_0(x) dx . \quad (1) \quad \boxed{\text{A}}$$

This is the expected value of the *observable* V in the *baseline* distribution f_0 . The *direct* or *baseline* estimator uses n samples $X_k \sim f_0$ and takes

$$\hat{A}_0 = \frac{1}{n} \sum_{k=1}^n V(X_k) , \quad X_k \sim f_0 . \quad (2) \quad \boxed{\text{be}}$$

The variance of the baseline estimator is

$$\text{var}_0(\hat{A}_0) = \frac{1}{n} \sigma_0^2 \quad (3) \quad \boxed{\text{Ef}}$$

$$\sigma_0^2 = \mathbb{E}_{f_0}[V(X)^2] - (\mathbb{E}_{f_0}[V(X)])^2$$

$$\sigma_0^2 = \int V(x)^2 f_0(x) dx - A^2 . \quad (4) \quad \boxed{\text{s20}}$$

Let $f(x, r)$ be a family of probability densities with $f_0(x) = f(x, 0)$. Expectation in the r density is

$$\mathbb{E}_r[W(X)] = \int W(x) f(x, r) dx . \quad (5) \quad \boxed{\text{Er}}$$

A *twisted* estimate of A uses the identities and notation

$$A = \int V(x) \frac{f_0(x)}{f(x, r)} f(x, r) dx \quad (6) \quad \boxed{\text{ta}}$$

$$= \int V(x) L(x, r) f(x, r) dx \quad (7) \quad \boxed{\text{tal}}$$

$$= \mathbb{E}_r[V(X) L(X, r)] \quad (8) \quad \boxed{\text{taE}}$$

$$L(x, r) = \frac{f_0(x)}{f(x, r)} \quad (9) \quad \boxed{\text{L}}$$

The twisted estimator of A uses n “twisted” samples, which are samples from $f_r(\cdot, r)$:

$$\hat{A}_r = \frac{1}{n} \sum_{k=1}^n V(X_k) L(X_k, r) \quad (10) \quad \boxed{\text{te}}$$

$$X_k \sim f(\cdot, r) , \quad \text{i.i.d. samples} \quad (11) \quad \boxed{\text{ts}}$$

The variance of the twisted estimator is

$$\begin{aligned}\text{var}(\widehat{A}_r) &= \frac{1}{n} \sigma_r^2 \\ \sigma_r^2 &= \text{E}_r[V(X)L(X, r)] \\ &= \text{E}_r[V(X)^2 L(X, r)^2] - A^2\end{aligned}\tag{12} \quad \boxed{\text{tv}}$$

The optimal r is the one that minimizes the twisted estimator variance (12). This project tries to find the optimal r using stochastic approximation via the Robbins Monro algorithm and stochastic sensitivity analysis.

A *stochastic sensitivity analysis* is a representation of the gradient of some expected value as the expected value of a different random variable. Suppose

$$u(r) = \text{E}_r[F(X, r)] .\tag{13} \quad \boxed{\text{ur}}$$

Here u depends on r both through the direct dependence of F on r and indirectly through the r dependence of the distribution of X . A sensitivity analysis (more properly, sensitivity “representation”) is a function $G(x, r)$ so that

$$u'(r) = \text{E}_r[G(X, r)] .\tag{14} \quad \boxed{\text{up}}$$

The G is not unique. In the present application, the target loss function to be minimized is (12) with the constant term ignored

$$u(r) = \text{E}_r[V(X)^2 L(X, r)^2] .\tag{15} \quad \boxed{\text{lf}}$$

The loss random variable is $F(x, r) = V(x)^2 L(x, r)^2$.

The loss function (15) may be written

$$u(r) = \int V(x)^2 \frac{f_0(x)^2}{f(x, r)^2} f(x, r) dx = \int V(x)^2 \frac{f_0(x)^2}{f(x, r)} dx .$$

This gives

$$u'(r) = - \int V(x)^2 \frac{f_0(x)^2}{f(x, r)^2} \partial_r f(x, r) dx\tag{16}$$

$$= - \int V(x)^2 \frac{f_0(x)^2}{f(x, r)^2} \frac{\partial_r f(x, r)}{f(x, r)} f(x, r) dx$$

$$= - \int V(x)^2 L(x, r)^2 S(x, r) f(x, r) dx$$

$$u'(r) = -\text{E}_r[V^2(X)L^2(X, r)S(X, r)] .\tag{17} \quad \boxed{\text{sa}}$$

These expressions use the *score function*

$$S(x, r) = \frac{\partial_r f(x, r)}{f(x, r)} = \partial_r \log(f(x, r)) .\tag{18} \quad \boxed{\text{S}}$$

This gives a sensitivity analysis of the form (14) with $G(x, r) = V^2(x)L^2(x, r)S(x, r)$.

Stochastic approximation/Robbins Monro

Gradient descent with “learning rate” parameters t_n is the algorithm

$$r_{n+1} = r_n - t_n \nabla_r u(r_n) .$$

The Robbins Monro algorithm is the same, except that the expected value ∇u is replaced by one sample of the sensitivity analysis random variable in (17)^{sa}

$$G(x, r) = V^2(x) L^2(x, r) S(x, r) . \quad (19) \quad \boxed{\text{G}}$$

The Robbins Monro gradient descent iteration is

$$r_{n+1} = r_n - t_n G(X_n, r_n) , \quad X_n \sim f(\cdot, r_n) . \quad (20) \quad \boxed{\text{RM}}$$

We fix the learning rate, $t_n = t$ for all t , unless we don't.

The *minibatch* variant uses m independent samples of G for each gradient descent step. For each n and r_n , you make m samples

$$X_{n,k} \sim f(\cdot, r_n) .$$

Then you compute the sample average of the m random “gradients”

$$\bar{G}_n = \frac{1}{m} \sum_{k=1}^m G(X_{n,k}, r_n) . \quad (21) \quad \boxed{\text{mb}}$$

Finally,

$$r_{n+1} = r_n - t_n \bar{G}_n .$$

Model problem example, some tests

The model problem is to evaluate

$$A = \mathbb{E}_{\mathcal{N}(0,1)}[X^4] = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} x^4 e^{-\frac{1}{2}x^2} = 3 . \quad (22) \quad \boxed{\text{A1}}$$

This has $f_0(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2}$ and $V(x) = x^4$. The importance sampling twisted densities are normal with mean zero and standard deviation $r \neq 1$

$$f(x, r) = \frac{1}{\sqrt{2\pi r^2}} e^{-\frac{1}{2r^2}x^2} .$$

The score function is

$$\begin{aligned} \partial_r \log \left(\frac{1}{\sqrt{2\pi r^2}} e^{-\frac{1}{2r^2}x^2} \right) &= \partial_r \left(-\log(r) - \frac{x^2}{2r^2} \right) \\ &= -\frac{1}{r} + \frac{x^2}{r^3} \\ S(x, r) &= \frac{1}{r} \left(\frac{x^2}{r^2} - 1 \right) . \end{aligned} \quad (23) \quad \boxed{\text{S_ex}}$$

The likelihood ratio $\frac{L}{f_0}$ is

$$L(x, r) = \frac{\frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2}}{\frac{1}{\sqrt{2\pi r^2}} e^{-\frac{1}{2r^2}x^2}} = r e^{-\frac{x^2}{2}(1-\frac{1}{r^2})}. \quad (24) \quad \boxed{\text{L_ex}}$$

Example 1, verify importance sampling

The module `m1_ex1.jl` shows that importance sampling (“twisted sampling”) can work in the sense of getting the right mean with a smaller variance. It takes f_0 to be the standard normal density and f_r as above. It computes A directly using n standard normal samples. The results are in [fig:m1_ex1](#).

The mean is correct using both methods, but the variance for the twisted estimator is smaller. The variance for the direct estimate is

$$\begin{aligned} \text{var}(V(X)) &= E[V(X)^2] - (E[V(X)])^2 \\ &= E[X^8] - (3)^2 \\ &= 7 \cdot 5 \cdot 3 - 9 \\ &= 96 \end{aligned}$$

The estimated variance (from the direct estimator) is close to this.

```
[[MacBook-Pro-2:~/desktop/projects/OptimalTwisting] jg% julia m1_ex1.jl
model 1, example 1, verify twisted sampling

using n = 5.0000e+07 samples and twisted standard deviationr = 2.00
direct estimate is 3.0003e+00, with sample variance 9.6287e+01
twisted estimate is 2.9994e+00, with sample variance 7.9255e+00
```

Figure 1: Example 1, verify that importance sampling can have the right expected value, 3 in this case, and reduce the variance, from 96 to about 8 in this case. The code is in `m1_ex1.jl`. [fig:m1_ex1](#)

Example 2, Verify score function sensitivity analysis

The module `m1_ex2.jl` demonstrates a sensitivity analysis formula. The target function is

$$u(r) = E[X^4] = 3r^4.$$

The sensitivity analysis formula is

$$u'(r) = 12r^3 = E[X^4 S(X, r)].$$

The score function is [S_ex](#) (23). [Figure 2](#) shows that the empirical change in u is accurately predicted by the sensitivity analysis formula, but only if dr is very small. [fig:m1_ex2](#)

```
[[MacBook-Pro-2:~/desktop/projects/OptimalTwisting] jg% julia m1_ex2.jl
model 1, example 2, test sensitivity analysis formulas
using 5.0000e+07 samples and dV(x) = x^4
```

```
baseline u0 = u(r0) = 3.0006e+00, computed sensitivity is 1.2002e+01
dr      u(r0+dr) (theory)      u(r+dr) (computed)      du/dr
2.0e-01      6.2208e+00      6.2221e+00      1.6107e+01
1.0e-01      4.3923e+00      4.3932e+00      1.3926e+01
5.0e-02      3.6465e+00      3.6473e+00      1.2933e+01
2.0e-02      3.2473e+00      3.2480e+00      1.2367e+01
1.0e-02      3.1218e+00      3.1225e+00      1.2184e+01
```

Figure 2: Example 2, verify that importance sampling can have the right expected value, 3 in this case, and reduce the variance, from 96 to about 8 in this case.

fig:m1_ex2

Example 3, Verify sensitivity analysis for variance

The module `m1_ex3.jl` verifies the sensitivity analysis formulas for variance given in (17) and (18) for the model problem. The variance is estimated as the empirical variance and the score function is (23). The results in Figure 3 show that the linear approximation using the gradient is quantitatively accurate only if dr is very small. This computation used a large number of samples: $n = 5 \cdot 10^8 =$ five hundred million samples. This would not work in Python.

```
[[MacBook-Pro-2:~/desktop/projects/OptimalTwisting] jg% julia m1_ex3.jl
model 1, example 3, test sensitivity analysis formulas
using 5.0000e+08 samples and dV(x) = x^4
```

```
baseline variance = 9.5990e+01, computed sensitivity is -8.4047e+02
dr      ssq(r+dr) (computed)      dssq/dr
1.0e-01      4.7204e+01      -4.8787e+02
1.0e-02      8.8118e+01      -7.8720e+02
5.0e-03      9.1924e+01      -8.1334e+02
2.0e-03      9.4331e+01      -8.2976e+02
1.0e-03      9.5155e+01      -8.3537e+02
```

Figure 3: Example 3, verify that importance sampling can have the right expected value, 3 in this case, and reduce the variance, from 96 to about 8 in this case.

fig:m1_ex3

Example 4, stochastic gradient descent

Here are experiments with Robbins Monro stochastic gradient descent using the sensitivity analysis (19) and iteration (20). The module `m1_ex4.jl` is the implementation. The starting point is always $r_0 = 1$. The optimal r is (see Figure 4) $r_* \approx 2.2$. Figure 4 plots ten independent optimization trajectories using a relatively large learning rate parameter. The results would be OK if the algorithm didn't have a tendency to take huge optimization steps away from the starting value $r \approx 1$. Figure 5 is a similar experiment but with a smaller learning rate, hopefully to avoid such large jumps.

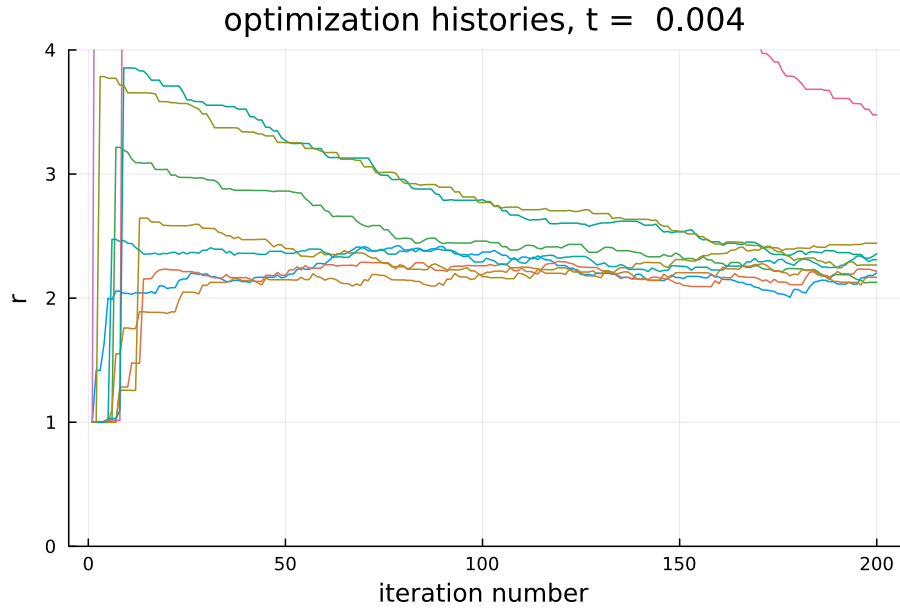


Figure 4: Example 4, plot with ten independent Robbins Monro trajectories, no minibatch, large learning rate. You can see that r trajectories approach an optimal r_* but do not converge to it because of noise in the Robbins Monro algorithm. This asymptotic noise is reduced by smaller t or (according to the theorem of Robbins and Monro) by letting $t_n \rightarrow 0$ as $n \rightarrow \infty$.

fig:m1_ex4_t4

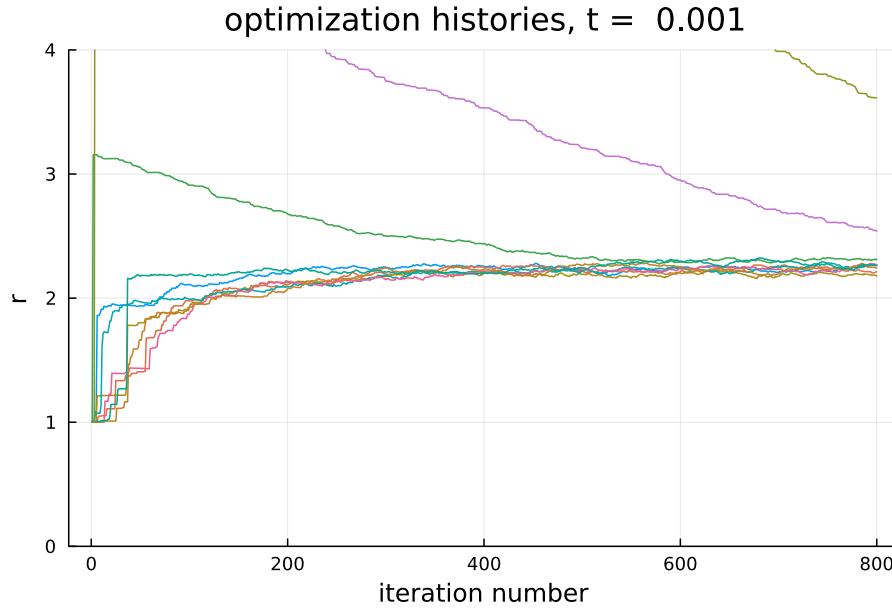


Figure 5: Example 4, plot with ten independent Robbins Monro trajectories, no minibatch, smaller learning rate. This too has large jumps from $r \approx 1$. Note that this experiment does 800 iterations as opposed to 200 from the large t experiment of Figure 4.

fig:m1_ex4_t1

You might hope that a bigger minibatch method $(\overset{\text{mb}}{21})$ would make single large gradient samples less disruptive. Figure

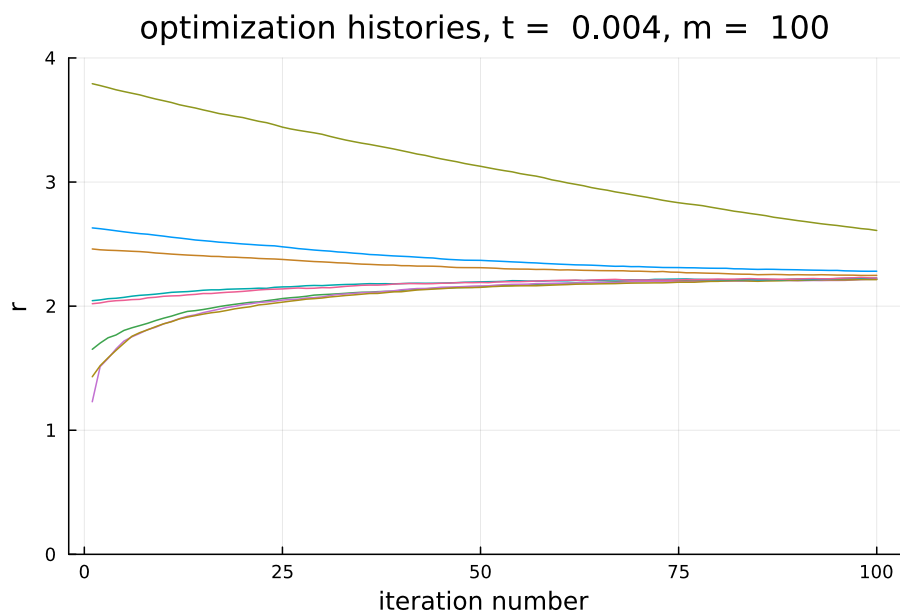


Figure 6: Example 4, plot with ten independent Robbins Monro trajectories, large learning rate and moderate ($m = 100$) batch size. Three of the 10 trajectories have clear disruptions, one of them rather large.

fig:m1_ex4_t4_m100

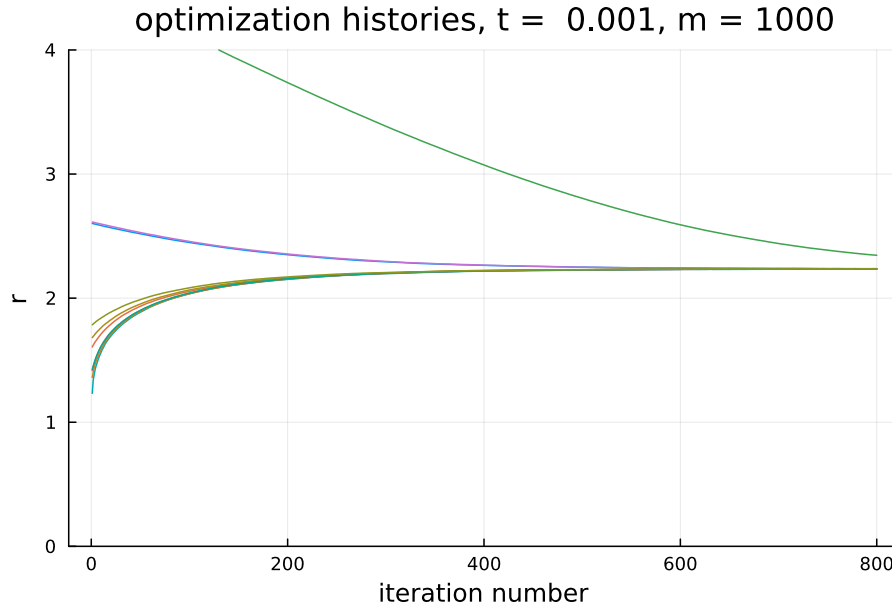


Figure 7: Example 4, plot with ten independent Robbins Monro trajectories, small learning rate and large ($m = 1000$) batch size, both of which are supposed to reduce disruptions. You can see that disruptions still happen. The large batch size makes the curves smoother.

fig:m1_ex4_t1_m1000

Example 5, plot the estimation variance

The module `m1_ex5.jl` plots the variance of the twisted estimator as a function of the twisting parameter r . The plot goes into the file `m1_ex5.pdf`.

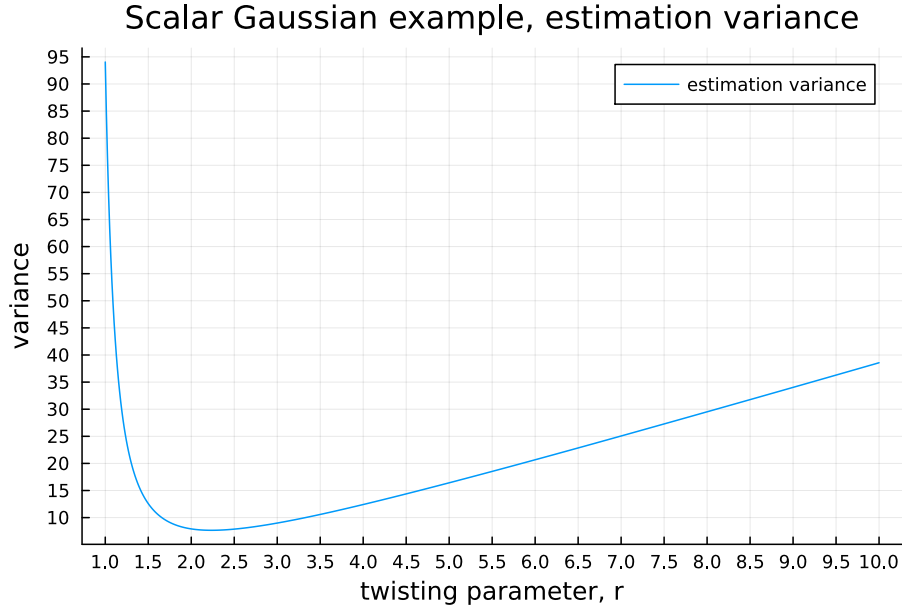


Figure 8: Example 5, plot the variance of the estimator as a function of the twisting parameter r . The minimizing r is a little bigger than 2 but the curve is pretty flat near the minimizer.

fig:m1_ex5

Model problem with a 1D “neural net”

The same model problem but a more general importance sampling strategy. You sample a Gaussian with mean zero and variance r^2 using $X = rZ$. If $Z \sim \mathcal{N}(0, 1)$, then $X \sim \mathcal{N}(0, r^2)$. Consider a more general transformation with function and inverse pair

$$x = \phi(z, \theta) \quad , \quad z = \psi(x, \theta) \quad . \quad (25)$$

NN1

Here, θ is a collection of parameters that determine the change of variables. The previous example has a one component $\theta = r$, with $\phi(z, r) = rz$ and $\psi(x, r) = \frac{1}{r}x$. We let p be the number of parameters, so

$$\theta = (\theta_1 \quad \cdots \quad \theta_p) \quad .$$

We suppose the change of variables is one to one, which may be hard to verify in practice and may be a reason this doesn’t generalize well?

The idea is to generate $Z \sim f_0(z)$, then take $X = \phi(Z, \theta)$. The PDF of X will be denoted $f(x, \theta)$. We hope that we can find a ϕ that gives a good f . The PDF of X is determined by

$$\Pr(x \leq X \leq x + dx) = f(x, \theta)dx \quad (26)$$

f

The base probability density is defined by

$$\Pr(z \leq Z \leq z + dz) = f_0(z)dz . \quad (27) \quad \boxed{\mathbf{f_0}}$$

We denote partial derivatives using subscripts. We write f_θ for the gradient of f with respect to the parameters θ . Thus

$$\begin{aligned} f_{0z}(z) &= \frac{\partial}{\partial z} f_0(z) \\ f_x(x, \theta) &= \frac{\partial}{\partial x} f(x, \theta) \\ \psi_\theta(x, \theta) &= \left(\frac{\partial}{\partial \theta_1} \psi(x, \theta), \dots, \frac{\partial}{\partial \theta_1} \psi(x, \theta) \right) \end{aligned}$$

We find the formula for f in terms of f_0 using informal calculus. If $x = \phi(z, \theta)$ and $X = \phi(Z, \theta)$, and if ϕ is one to one in its first argument, then $dx = \phi_z(z, \theta)dz$. This, together with (26) and (27), this gives

$$f(x, \theta) \phi_z(z, \theta) dz = f_0(z) dz .$$

Therefore

$$f(x, \theta) = \frac{1}{\phi_z(z, \theta)} f_0(z) . \quad (28) \quad \boxed{\mathbf{ff_0}}$$

It may be convenient to write this in different ways using the change of variable functions (25), such as

$$\begin{aligned} f(\phi(z, \theta), \theta) &= \frac{1}{\phi_z(z, \theta)} f_0(z) \\ f(x, \theta) &= \frac{1}{\phi_z(\psi(x, \theta), \theta)} f_0(\psi(x, \theta)) \\ f(x, \theta) &= \psi_x(x, \theta) f_0(\psi(x, \theta)) \\ &\text{etc.} \end{aligned}$$

You can repeat the derivation of the score function sensitivity formulas (17)^{sa} and (18)^s with a multi-component θ instead of r . The results are

$$u(\theta) = E_\theta [V(X)^2 L^2(X, \theta)] \quad (29) \quad \boxed{\mathbf{mlf}}$$

$$u_\theta = E_\theta [V(X)^2 L^2(X, \theta) S(X, \theta)] \quad (30) \quad \boxed{\mathbf{msa}}$$

$$S(x, \theta) = - \frac{f_\theta(x, \theta)}{f(x, \theta)} . \quad (31) \quad \boxed{\mathbf{mS}}$$

The gradient in (30)^{msa} is

$$u_\theta(\theta) = \nabla_\theta u(\theta) = \left(\frac{\partial}{\partial \theta_1} u(\theta), \dots \right)$$

The score function (31)^{ms} is a vector of the same dimension.

Third try

Start with

$$f(x, \theta) = \frac{f_0(\psi(x, \theta), \theta)}{\phi_z(\psi(x, \theta), \theta)} .$$

The score function is

$$\begin{aligned} S(x, \theta) &= \frac{\partial_\theta f(x, \theta)}{f(x, \theta)} \\ S(x, \theta) &= \frac{\partial_\theta \frac{f_0(\psi(x, \theta))}{\phi_z(\psi(x, \theta), \theta)}}{\frac{f_0(\psi(x, \theta))}{\phi_z(\psi(x, \theta), \theta)}} \end{aligned}$$

In these calculations, f_0 depends on θ only indirectly through the dependence of $z = \psi(x, \theta)$. On the other hand, ϕ_z depends on θ both directly, as $\phi_z(z, \theta)$, and indirectly through the dependence of z on θ . There is one component of S for each component of θ , but I write θ and ∂_θ instead of θ_k and ∂_{θ_k} here. In the code, you apply the formula for S for each component of θ .

We get an expression for S in terms of ϕ , derivatives of ϕ , f_0 and its derivatives. These quantities are available in the code. First comes the algebra, then comes pictures and discussion to support the algebra. The numerator above is

$$\partial_\theta \frac{f_0(\psi(x, \theta))}{\phi_z(\psi(x, \theta), \theta)} = \frac{\partial_\theta f_0(\psi(x, \theta))}{\phi_z(\psi(x, \theta), \theta)} - \frac{f_0(\psi(x, \theta)) \partial_\theta \phi_z(\psi(x, \theta), \theta)}{\phi_z(\psi(x, \theta), \theta)^2}$$

Dividing by the denominator gives

$$S(x, \theta) = \frac{\partial_\theta f_0(\psi(x, \theta))}{f_0(\psi(x, \theta))} - \frac{\partial_\theta \phi_z(\psi(x, \theta), \theta)}{\phi_z(\psi(x, \theta), \theta)} = S_I(x, \theta) + S_{II}(x, \theta)$$

The separate parts are

$$S_I(x, \theta) = \frac{\partial_\theta f_0(\psi(x, \theta))}{f_0(\psi(x, \theta))}$$

and

$$S_{II}(x, \theta) = -\frac{\partial_\theta \phi_z(\psi(x, \theta), \theta)}{\phi_z(\psi(x, \theta), \theta)}$$

For S_I the chain rule gives

$$\partial_\theta f_0(\psi(x, \theta)) = f_{0z}(z) \partial_\theta \psi(x, \theta) .$$

To be clear, the partial derivative with respect to θ is the one holding x fixed. We write z for $\psi(x, \theta)$ once no more differentiation is required. We calculate $\partial_\theta \psi(x, \theta)$ using implicit differentiation of the identity $x \xrightarrow{\psi} z \xrightarrow{\phi} x$:

$$\begin{aligned} x &= \phi(\psi(x, \theta), \theta) \\ \partial_\theta x &= 0 = \phi_z(\psi(x, \theta), \theta) \partial_\theta \psi(x, \theta) + \phi_\theta(\psi(x, \theta), \theta) \\ \partial_\theta \psi(x, \theta) &= -\frac{\phi_\theta(z, \theta)}{\phi_z(z, \theta)} . \end{aligned}$$

Therefore

$$S_I(x, \theta) = -\frac{f_{0z}(z)}{f_0(z)} \frac{\phi_\theta(z, \theta)}{\phi_z(z, \theta)}. \quad (32) \quad \boxed{\text{SI}}$$

For S_{II} , we calculate

$$\begin{aligned} \partial_\theta \phi_z(\psi(x, \theta), \theta) &= \phi_{zz}(z, \theta) \partial_\theta \psi(x, \theta) + \phi_{z\theta}(z, \theta) \\ &= -\phi_{zz}(z, \theta) \frac{\phi_\theta(z, \theta)}{\phi_z(z, \theta)} + \phi_{z\theta}(z, \theta) \end{aligned}$$

Therefore,

$$S_{II}(x, \theta) = \frac{\phi_{zz}(z, \theta) \phi_\theta(z, \theta)}{\phi_z(z, \theta)^2} - \frac{\phi_{z\theta}(z, \theta)}{\phi_z(z, \theta)}. \quad (33) \quad \boxed{\text{SII}}$$

As a sanity check, S represents the change in the PDF $f(x, \theta)$ when θ changes with x fixed. Since $f(x, \theta) = f_0(z)/\phi_z(z, \theta)$, the change in f is due either to a change in f_0 or a change in ϕ_z . The term S_I is the part corresponding to a change in f_0 which is due to a change in z . A diagram shows that if ϕ_θ and ϕ_z are positive, then increasing θ decreases z . Thus, if f_{0z} is positive, S_I should be negative.

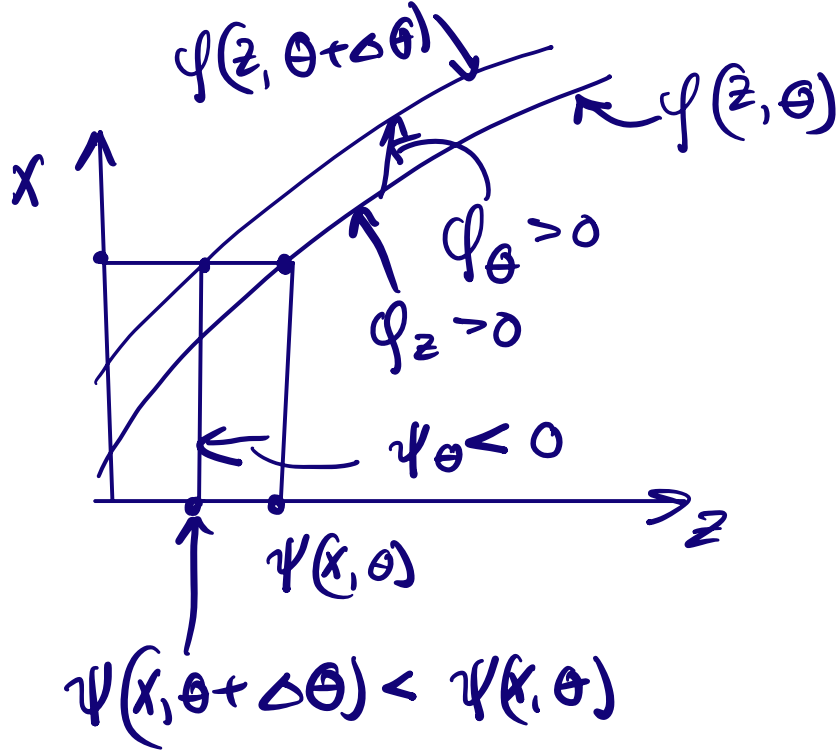


Figure 9: Diagram showing $S_I < 0$ if $\phi_z > 0$, $\phi_\theta > 0$ and $f_z > 0$.

fig:S_I

the example

The “neural net” defines the change of variables $x = \phi(z, \theta)$. For this problem, it seems good to under-sample the region near $x = 0$ (relative to the standard normal) by making ϕ' large there. In the notation of the previous example, this is using a large r , at least for z values that are not large. If you use a fixed r everywhere, this leads to too many very large samples X_j where $V^2(X_j)$ is so large that the method “goes nuts” (creates a very large step even with a small learning rate). A possible partial cure would be to reduce $\phi'(z)$ when z is not so small. We try to do this using a smooth transition between large and small slope using a smooth Relu activation function (one of the suggestions in the Wikipedia page)

$$\sigma(t) = \log(1 + e^t) . \quad (34) \quad \boxed{\text{sig}}$$

The unsmoothed Relu function is

$$\sigma_0(t) = t_+ = \begin{cases} t & \text{if } t \geq 0 \\ 0 & \text{if } t \leq 0 \end{cases}$$

The smoothed one ^(sig)(32) has the same large $\pm t$ behavior, which is

$$\sigma(t) = \begin{cases} t + O(e^{-t}) & \text{as } t \rightarrow \infty \\ O(e^{-|t|}) & \text{as } t \rightarrow -\infty \end{cases}$$

The derivative is

$$\sigma'(t) = \frac{e^t}{1 + e^t} .$$

This has $\sigma'(t) \rightarrow 0$ as $t \rightarrow -\infty$ and $\sigma'(t) \rightarrow 1$ as $t \rightarrow \infty$. The second derivative is

$$\sigma''(t) = \frac{e^t}{(1 + e^t)^2} .$$

This is positive for all t , which means that σ' transitions monotonically from 0 to 1 and that $\sigma(t)$ is a convex function of t .

This suggests a function like

$$\phi_+(z) = az - b\sigma((z - c)/d) . \quad (35) \quad \boxed{\text{php}}$$

The four scalar parameters would be

$$\theta = (\theta_1, \theta_2, \theta_3, \theta_4) = (a, b, c, d) .$$

We imagine that the coefficients are all positive: a (the slope at 0), $b > 0$ (so the slope decreases as z increases), c (the center of the transition zone from large to smaller slope, and d (the width of the slope transition zone). The slope at large z (treating $\sigma(t) = t$ for large t) is

$$\phi'_+(z) \rightarrow a - \frac{b}{d} , \quad \text{as } z \rightarrow \infty .$$

Thus, $\phi'_+(z) > 0$ for all z if $a > \frac{b}{d}$. To be clear, $\phi'_+(0) > a$, but only by a small amount if 0 is far from the slope transition zone.

The model problem is symmetric in x , so it makes sense to have a symmetric distribution, which would be $f(x) = f(-x)$. This happens if $f_0(z)$ is a symmetric function of z (we take standard normal) and $\phi(z, \theta)$ is a skew symmetric function of z :

$$\phi(-z, \theta) = -\phi(z, \theta) .$$

The formula (33) is not skew symmetric, so we skew symmetrize by taking

$$\phi(z, \theta) = \phi_+(z, \theta) - \phi_+(-z, \theta) . \quad (36) \quad \boxed{\text{ss}}$$

Formulas for derivatives of ϕ

$$\phi(z, \theta) = 2az - b \left[\sigma\left(\frac{z-c}{d}\right) - \sigma\left(\frac{-z-c}{d}\right) \right] .$$

The parameters are $\theta = (a, b, c, d)$. The derivatives are

$$\partial_a \phi(z, \theta) = 2z \quad (37) \quad \boxed{\text{phi_a}}$$

$$\partial_b \phi(z, \theta) = -\sigma\left(\frac{z-c}{d}\right) + \sigma\left(\frac{-z-c}{d}\right)$$

$$\partial_c \phi(z, \theta) = \frac{b}{d} \left[\sigma'\left(\frac{z-c}{d}\right) - \sigma'\left(\frac{-z-c}{d}\right) \right]$$

$$\partial_d \phi(z, \theta) = \frac{b}{d^2} \left[(z-c)\sigma'\left(\frac{z-c}{d}\right) + (z+c)\sigma'\left(\frac{-z-c}{d}\right) \right]$$

$$\partial_d \phi(z, \theta) = \frac{b}{d} \left[\frac{z-c}{d} \sigma'\left(\frac{z-c}{d}\right) + \frac{z+c}{d} \sigma'\left(\frac{-z-c}{d}\right) \right]$$

For coding, we use notations for the arguments of the activation function σ :

$$t_+ = \frac{z-c}{d}$$

$$t_- = -\frac{z+c}{d}$$

The gradient formulas above become

$$\partial_b \phi(z, \theta) = -\sigma(t_+) + \sigma(t_-) \quad (38) \quad \boxed{\text{phi_b}}$$

$$\partial_c \phi(z, \theta) = \frac{b}{d} [\sigma'(t_+) - \sigma'(t_-)] \quad (39) \quad \boxed{\text{phi_c}}$$

$$\partial_d \phi(z, \theta) = \frac{b}{d} [t_+ \sigma'(t_+) - t_- \sigma'(t_-)] \quad (40) \quad \boxed{\text{phi_d}}$$

We also need the second derivatives with respect to z and θ

$$\begin{aligned}
\partial_z \partial_a \phi(z, \theta) &= 2 & (41) \quad \boxed{\text{phi_za}} \\
\partial_z \partial_b \phi(z, \theta) &= -\frac{1}{d} \left[\sigma' \left(\frac{z-c}{d} \right) + \sigma' \left(\frac{-z-c}{d} \right) \right] \\
\partial_z \partial_c \phi(z, \theta) &= \frac{b}{d^2} \left[-\sigma'' \left(\frac{z-c}{d} \right) + \sigma'' \left(\frac{-z-c}{d} \right) \right] \\
\partial_z \partial_d \phi(z, \theta) &= \frac{b}{d^2} \left[\sigma' \left(\frac{z-c}{d} \right) + \sigma' \left(\frac{-z-c}{d} \right) + \frac{z-c}{d} \sigma'' \left(\frac{z-c}{d} \right) + \frac{z+c}{d} \sigma'' \left(\frac{-z-c}{d} \right) \right]
\end{aligned}$$

With the $t_{\pm}$ notation, these are

$$\partial_z \partial_b \phi(z, \theta) = -\frac{1}{d} [\sigma'(t_+) + \sigma'(t_-)] \quad (42) \quad \boxed{\text{phi_zb}}$$

$$\partial_z \partial_c \phi(z, \theta) = \frac{b}{d^2} [\sigma''(t_+) + \sigma''(t_-)] \quad (43) \quad \boxed{\text{phi_zc}}$$

$$\partial_z \partial_d \phi(z, \theta) = \frac{b}{d^2} [\sigma'(t_+) + \sigma'(t_-) + t_+ \sigma''(t_+) + t_- \sigma''(t_-)] \quad (44) \quad \boxed{\text{phi_zd}}$$